SMARTDESK_AI

Capstone 5 Project

ID	Phase	Task Description
1	LEARN	Learn what Python is and set it up: Watch a beginner YouTube video on Python (search "Python for absolute beginners").
1a	SETUP	Then go to python.org and download Python 3.11 for your computer. Run the installer and check the box that says "Add Python to PATH" before clicking Install.
2	LEARN	Learn what a terminal or command prompt is: On Windows press the Windows key and type "cmd" then press Enter. On Mac press Command+Space and type "terminal". Practice typing simple commands like "dir" (Windows) or "ls" (Mac) to see files in a folder.
3	LEARN	Learn what VS Code is
3a	SETUP	Install it: Go to code.visualstudio.com and download Visual Studio Code. This is the app you will write all your code in. Install it and open it. It is free and beginner friendly.
4	LEARN	Learn what a Python virtual environment is: Watch a short YouTube video searching "Python virtual environment beginner". A virtual environment is like a clean backpack just for your project so tools from one project do not mess up another.
5	LEARN	Learn what an API key is: Read a short article by searching "what is an API key explained simply". Think of it like a password that lets your program talk to another service like OpenAI or Jira. You never share it publicly.
6	LEARN	Learn what RAG means: Search YouTube for "retrieval augmented generation explained simply". RAG means your AI looks up information from YOUR documents before answering instead of just guessing. This is the heart of SmartDesk AI.
7	LEARN	Learn what a vector database is: Search YouTube for "vector database explained for beginners". Think of it like a smart filing cabinet that stores your documents in a way that makes searching by meaning possible instead of just exact words.
8	LEARN	Learn what Jira is and create a free account: Go to atlassian.com/software/jira and sign up for a free Jira Cloud account using your email. Jira is a popular tool companies use to track problems and tasks. Your SmartDesk AI will create and read tickets here.
9	LEARN	Learn what GitHub is and create a free account: Go to github.com and sign up for a free account. GitHub is like Google Drive but for code. You will store and submit your entire project here. Watch a YouTube video searching "GitHub for beginners".
10	LEARN	Learn what an LLM is: Search "what is a large language model for beginners". An LLM is the AI brain (like ChatGPT) that reads your documents and writes helpful answers. You will connect your project to one of these.
11	SETUP	Create an OpenAI account and get an API key: Go to platform.openai.com and sign up. Click your profile icon in the top right then click "API Keys" then click "Create new secret key". Copy it and save it in a notepad file on your desktop temporarily. You will move it to a safe place in a later step.
12	SETUP	Create your project folder: Open VS Code. Click "File" then "Open Folder". Create a new folder on your Desktop called "SmartDesk_AI" and open it. This is where every file for your project will live.
13	SETUP	Create a virtual environment for your project: In VS Code click "Terminal" at the top menu then "New Terminal". A black panel opens at the bottom. Type this exactly and press Enter: python -m venv venv. Then activate it by typing: venv\Scripts\activate (Windows) or source venv/bin/activate (Mac). You will see (venv) appear on the left of your terminal line.
14	SETUP	Create your environment variables file to store secrets safely: 1) In VS Code right-click in the left file panel and click "New File". 2) Name it .env. 3) Inside type: OPENAI_API_KEY=paste_your_key_here. Never share this file. This keeps your API key out of your code.
15	SETUP	Install the python-dotenv library to read your .env file: In your terminal type: pip install python-dotenv and press Enter. This small tool lets your code read the secret keys from the .env file automatically.
16	SETUP	Create your GitHub repository and connect it to your folder: On github.com click the green "New" button. Name it SmartDesk_AI. Set it to Public. Copy the repo link. In VS Code terminal type: git init then git remote add origin YOUR_LINK_HERE. This connects your folder to GitHub.
17	SETUP	Create a .gitignore file to protect your secrets: In VS Code create a new file called .gitignore. On the first line type: .env. This tells GitHub to never upload your secret keys file. Very important step.

SMARTDESK_AI

Capstone 5 Project

ID	Phase	Task Description
18	SETUP	Create a Jira project for SmartDesk tickets: Log in to your Jira account. Click "Create project". Choose "Scrum" or "Kanban". Name it SmartDesk. This is where your AI agent will create and check support tickets later.
19	SETUP	Get your Jira API token: In Jira click your profile picture in the top right corner then click "Manage Account". Click the "Security" tab. Click "Create and manage API tokens". Click "Create API token". Name it SmartDesk and copy the token. Save it in your .env file as: JIRA_API_TOKEN=paste_here
20	SETUP	Add Jira connection details to your .env file: Open your .env file and add three new lines: JIRA_EMAIL=your_jira_email@email.com and JIRA_SERVER=https://yourname.atlassian.net and JIRA_PROJECT_KEY=the short code shown on your Jira project (example: SD).
21	KNOWLEDGE BASE	Learn about the knowledge base requirement: Re-read Section 2.1 and Section 3 of the capstone document. Your knowledge base needs at least 30 to 50 question and answer pairs covering IT help topics and HR policy topics. This is the information your AI will use to answer questions.
22	KNOWLEDGE BASE	Download a free HR dataset from Hugging Face: Go to huggingface.co/datasets/strova-ai/hr-policies-qa-dataset. Click the "Files and versions" tab. Download the CSV or JSON file. Save it inside a new folder called "knowledge_base" inside your SmartDesk_AI project folder.
23	KNOWLEDGE BASE	Generate synthetic IT support documents using ChatGPT: Go to chatgpt.com. Paste this prompt: "You are the IT Admin at AcmeCorp a 300 employee company. Write an internal guide covering: password reset steps VPN setup for Windows and Mac MFA setup and email setup on mobile. Write in simple professional language for employees." Copy the result into a new file in your knowledge_base folder called it_support_guide.md.
24	KNOWLEDGE BASE	Generate HR policy documents using ChatGPT: Go to chatgpt.com. Paste this prompt: "You are the HR Manager at AcmeCorp. Write a Leave Policy document covering casual leave sick leave earned leave and work from home policy. Include how to apply and approval steps. Keep it simple and professional." Save the result as hr_leave_policy.md in your knowledge_base folder.
25	KNOWLEDGE BASE	Generate Q&A pairs from your documents using ChatGPT: Go to chatgpt.com. Paste your it_support_guide.md content and add this instruction: "Generate 15 realistic employee questions and answers from this document. Output as JSON with fields: question answer category." Save the output as it_qa.json in your knowledge_base folder. Repeat for the HR policy file and save as hr_qa.json.
26	KNOWLEDGE BASE	Add deliberate gaps to your knowledge base: Using ChatGPT write a short list of 5 topics NOT covered in your knowledge base such as monitor flickering office parking and printer jams. Save this list as out_of_scope_topics.txt. These gaps will test your AI's ability to say "I don't know" and create a ticket instead.
27	KNOWLEDGE BASE	Review and clean your knowledge base files: Open each JSON and Markdown file in VS Code. Make sure all entries look realistic and professional. Replace any placeholder names with AcmeCorp. Aim for at least 30 total Q&A pairs across all files before moving on.
28	RAG PIPELINE	Install the core RAG libraries: In your VS Code terminal with venv activated type and press Enter: pip install openai chromadb langchain langchain-openai langchain-community. Wait for all packages to finish installing. These are the building blocks of your RAG pipeline.
29	RAG PIPELINE	Learn what chunking means in RAG: Search YouTube for "text chunking for RAG beginners". Chunking means cutting your long documents into smaller pieces before storing them. Smaller pieces help the AI find the most relevant part of a document faster.
30	RAG PIPELINE	Create the indexing script to load your documents into ChromaDB: In VS Code create a new file called index_knowledge_base.py. This script will read all your JSON and Markdown files from the knowledge_base folder cut them into chunks create embeddings using OpenAI and store them in ChromaDB. Ask your instructor or use the program materials for the exact code pattern covered in your coursework.
31	RAG PIPELINE	Run the indexing script and verify it works: In your terminal type: python index_knowledge_base.py. Watch for any red error messages. If it runs without errors your documents are now stored in ChromaDB. You should see a new folder called chroma_db appear in your project folder.
32	RAG PIPELINE	Test your retrieval pipeline with sample queries: Create a new file called test_retrieval.py. Write a small script that takes a question like "How do I reset my password?" sends it to ChromaDB and prints the top 3 matching chunks it finds. Run it and check if the results make sense. This proves your retrieval is working before you build the full agent.
33	RAG PIPELINE	Add confidence threshold logic to your retrieval: In your retrieval code add a check: if the best matching chunk has a similarity score below 0.75 the system should return a flag that says "not found" instead of a weak answer. This is the escalation trigger described in Section 2.1.3 of the capstone document.
34	RAG PIPELINE	Build the RAG answer function: Create a new file called rag_chain.py. This function takes a user question retrieves the top matching chunks from ChromaDB and sends both the question and the chunks to OpenAI GPT to generate a grounded answer. The system prompt must tell the AI to ONLY use the provided context and say "I don't have enough information" if the context does not help.

ID	Phase	Task Description
35	JIRA INTEGRATION	Install the Jira Python library: In your terminal type: pip install jira and press Enter. This library lets your Python code talk directly to your Jira account to create and read tickets without needing a browser.
36	JIRA INTEGRATION	Create a Jira connection test script: Create a file called test_jira_connection.py. Write code that loads your JIRA_EMAIL JIRA_API_TOKEN and JIRA_SERVER from the .env file and connects to Jira. Print your Jira account display name to confirm the connection works. Run it and check the output.
37	JIRA INTEGRATION	Build the ticket creation function: Create a file called jira_tools.py. Write a function called create_ticket that accepts employee_email summary description and category as inputs and creates a new Jira issue in your SmartDesk project. The function should return the new ticket ID like SD-1 so you can give it to the employee.
38	JIRA INTEGRATION	Build the ticket status function: In the same jira_tools.py file write a second function called get_ticket_status that accepts an employee email as input searches Jira for all issues where the reporter email matches and returns a list of tickets with their current status and any comments.
39	JIRA INTEGRATION	Test ticket creation manually: Create a file called test_create_ticket.py. Call your create_ticket function with fake test data. Run it. Then log in to your Jira account in the browser and check that the ticket actually appeared in your SmartDesk project. This confirms the write operation works end to end.
40	JIRA INTEGRATION	Test ticket status lookup manually: Create a file called test_get_status.py. Call your get_ticket_status function using the email you used when creating the test ticket. Run it and confirm the ticket details print correctly in the terminal. This confirms the read operation works end to end.
41	AGENT	Learn what agent orchestration means: Search YouTube for "LangGraph agent tutorial beginner". The orchestrator is the brain that decides: should I answer from the knowledge base OR ask for more info and create a ticket OR look up a ticket status? It routes the conversation to the right tool.
42	AGENT	Install LangGraph: In your terminal type: pip install langgraph and press Enter. LangGraph helps you build agents that can follow decision paths like a flowchart making it easier to manage the three flows in your capstone project.
43	AGENT	Design your agent flowchart on paper first: Before writing any code draw a simple flowchart on paper. Box 1: Employee sends a message. Box 2: Is this a ticket status question? Box 3: Is this answerable from KB? Box 4: Escalate to ticket creation. Drawing this first makes coding the logic much easier.
44	AGENT	Build the intent detection function: Create a file called agent.py. Write a function called detect_intent that takes the user message and uses OpenAI to classify it as one of three types: KB_QUERY or CREATE_TICKET or CHECK_STATUS. This is the router that sends the conversation down the right path.
45	AGENT	Build the full agent conversation loop: In agent.py write the main agent function that uses detect_intent to route the message then calls the right function: rag_chain for KB answers or jira_tools for ticket work. Keep a session memory dictionary that stores the employee email once provided so you never ask twice in the same conversation.
46	AGENT	Add the human-in-the-loop confirmation step for ticket creation: In your ticket creation flow add a step where the agent prints a summary of the ticket it is about to create and asks the employee "Shall I go ahead? Type yes or no." Only call create_ticket if the employee types yes. This is a required step in Section 2.2.2.
47	AGENT	Add graceful handling for out-of-scope questions: In your agent add an else branch for when intent is unclear or the RAG score is too low. The agent should say something like "I'm not sure I can answer that. Would you like me to create a support ticket for you?" Never let the agent go silent or crash on an unexpected question.
48	AGENT	Add error handling for API failures: Wrap your OpenAI calls and Jira calls in try and except blocks. If an API call fails print a friendly message like "I'm having trouble connecting right now. Please try again in a moment." This is required in Section 4.3 and worth marks in the Error Handling category.
49	TESTING	Run the Flow A test scenario from the capstone document: Open your terminal and run your agent. Type "How do I reset my password?" Confirm the agent returns a correct answer from your knowledge base without making anything up. If it hallucinates or returns nothing go back and fix your RAG chain or knowledge base.
50	TESTING	Run the Flow B test scenario from the capstone document: Type a question your knowledge base does NOT cover such as "My monitor keeps flickering." Confirm the agent says it cannot find the answer offers to create a ticket asks for your email shows a summary and creates the ticket only after you say yes. Check Jira to confirm the ticket appeared.
51	TESTING	Run the Flow C test scenario from the capstone document: Type "What is the status of my ticket?" Confirm the agent asks for your email then retrieves and displays the ticket you created in Flow B including its current status. If multiple tickets exist confirm it lists all of them.

ID	Phase	Task Description
52	TESTING	Test edge cases: Try sending blank messages rude messages questions in a different language and very long messages. Confirm your agent does not crash. It should always respond politely even if it cannot help. Fix any crashes you find.
53	TESTING	Test that no secrets are hard-coded: Search your entire project folder using VS Code search (Ctrl+Shift+F on Windows) for your actual API key text. If it appears anywhere except the .env file remove it immediately and load it from the environment variable instead.
54	DOCUMENT	Create your README.md file: In VS Code create a new file called README.md. Write sections for: Project Overview. Setup Instructions step by step. How to populate the knowledge base. How to run the agent. List of all environment variables needed. Example conversation flows.
55	DOCUMENT	Create your architecture diagram: Use draw.io at app.diagrams.net which is free. Draw a simple box-and-arrow diagram showing: Employee Input goes to Agent. Agent goes to ChromaDB. Agent goes to OpenAI. Agent goes to Jira. Save it as architecture_diagram.png and add it to your README.md.
56	DOCUMENT	Record your demo video: Use a free screen recorder like Loom at loom.com. Record yourself running all three flows: a KB answer a ticket creation and a ticket status check. Keep it between 3 and 5 minutes. Speak out loud explaining what is happening. Save the link for submission.
57	SUBMISSION	Do a final self-assessment using the rubric in Section 11: Open the capstone document and go to Section 11. Go through every row of the self-assessment grid honestly. For any row marked "Needs Work" or "Missing" go back and fix it before submitting. This is the same rubric your evaluator will use.
58	SUBMISSION	Push all your code to GitHub: In your VS Code terminal type: git add . then git commit -m "Final SmartDesk AI submission" then git push origin main. Go to github.com and open your repository to confirm all files are visible including your README and architecture diagram but NOT your .env file.
59	SUBMISSION	Create a requirements.txt file listing all your libraries: In your terminal type: pip freeze > requirements.txt and press Enter. This creates a file listing every library your project needs so anyone who clones your repo can install everything with one command: pip install -r requirements.txt.
60	SUBMISSION	Final checklist before submitting: Confirm all of these are true: GitHub repo is public or evaluator is added as collaborator. README.md is complete. Architecture diagram is included. Knowledge base files are in the repo. .env file is NOT in the repo. All three flows work when you run the agent fresh. requirements.txt is present. Demo video link is in README.